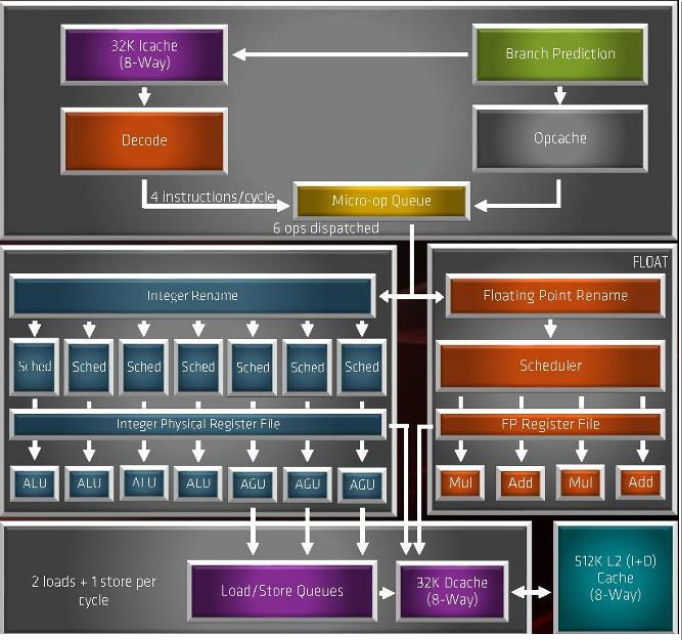


"Central Processing Unit", or CPU, generally consists of : Multiple processing units (cores) // Multiple memory levels (registers, L1, L2, L3, RAM) // Multiple execution ports in each core (ALUs, vector units) // Vector units for SIMD parallelism (AVX2, AVX512, Arm Neon, ...) // Simultenous execution of multiple threads (2-4) // Capable of exploiting the instruction level parallelism (ILP) with micro-op buffer, instruction reordering, register renaming, etc. // A good portion of hardware is dedicated to managing ILP + cache

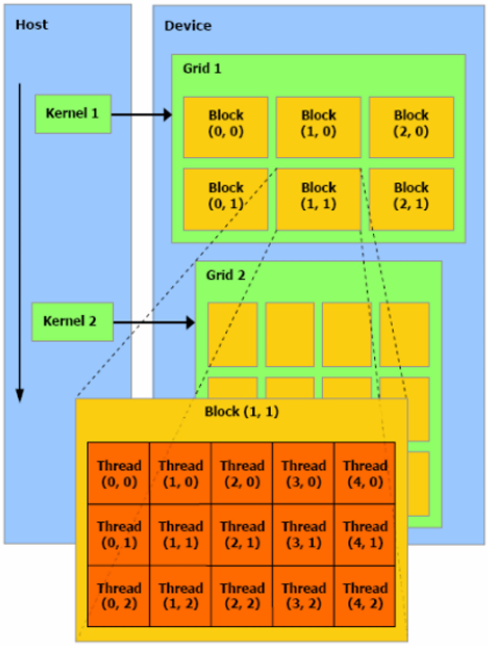
← Zen 2

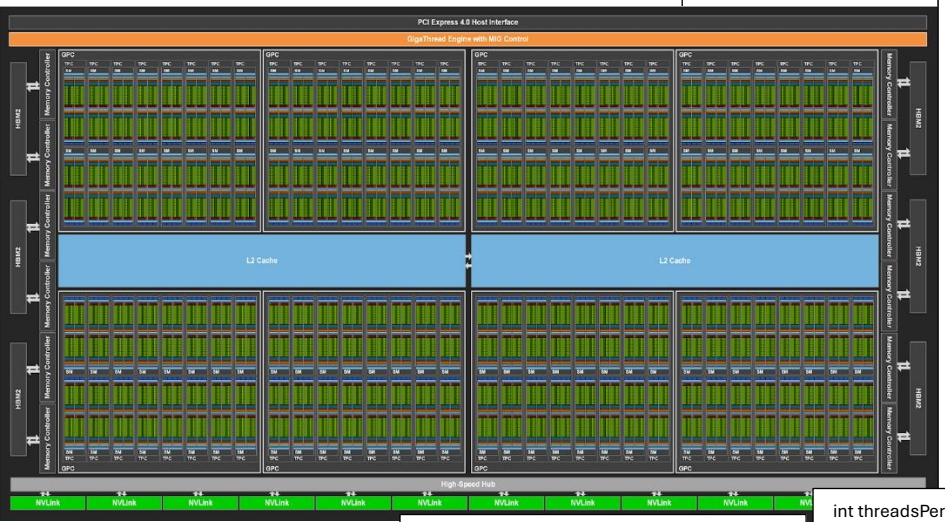
"Graphical Processing Unit", or GPU, is a vector processing unit consisting of

Multiple execution units (streaming multiprocessor s SMs) // Multiple memory levels (registers, shared memory, L1, L2, RAM) // Multiple vector units (1-8) that are wide (16-32 floats) in each SM. // Simultaneous execution of thousands of threads // Giant register table (65K) // Very fast context switching among threads, since threads reside in SMs simultenously // Most of the hardware is dedicated to vector compute units, very high performace potential. // Parallelization requires some effort

← nvidia ampere

Today, we already exceeded the exaflop scale (1018 f loating point operations per second, or flops/s)





^GPU kernel execution by blocks of threads^

GPU programming is adapted to the single-instruction-multiple-thread (SIMT) model.

Each GPU consists of multiple identical processors (like CPU cores) colled streaming multiprocessors (SM). // Each SM has multiple smaller cores, and each core can execute a thread simultenously (to come). // There is a kernel (GPU function) to execute by all threads on SM cores. // A block is the execution of a kernel on an SM. // Computation on a block differentiates itself from others by an identifier-blockIdx.x

The CPU launches the execution of a kernel with a set of GPU threads :

Identical threads (executing the same code!) // Threads organized into blocks (of size 32-1024) // Each identical block runs on an SM // Blocks organized into grids and distributed across all SMs // You need to launch sufficient number of blocks and threads so that the entire iteration domain of the problem is covered // Threads within the same block can share resources and communicate/synchronize (Threads in different blocks cannot!)

```
#include <cstdio>
#include "cuda.h"

__global__ void helloWorld()
{
  printf("Hello from block %d/%d\n",
      blockIdx.x, gridDim.x);
}

int main(int argc, char **argv)
{
  helloWorld<<<3, 1>>>();
  cudaDeviceSynchronize();
  return 0;
}
```

cuda.h provides necessary functions. // __global__ specifies the definition of a GPU kernel (otherwise it is a CPU function by default) // Defines blockIdx.x and gridDim.x // <<3,1>> creates 3 blocks (each having 1 thread, more to come) that execute the same code asynchronously. // Each block has an identifier between 0 et 3 (or P −1 if we launch the kernel with P blocks) // blockIdx.x is predefined and provides the block identifier in a GPU kernel // gridDim.x is predefined and provides the number of blocks used in a GPU kernel.

int threadsPerBlock = 256; // int numBlocks = N / threadsPerBlock; // kernelGPU<<>>(arg1, arg2, ...)

A qualifier is a key wordthatdifferentiates CPU/GPUfunctionsandvariables ina CUDAprogra

	__device__	__host__ (default)	__global__
Fonctions	Appel sur GPU Exec sur GPU	Appel sur CPU Exec sur CPU	Appel sur CPU Exec sur GPU
	__device__	__constant__	__shared__
Variables	Mémoire globale GPU Durée de vie de l'application Accessible par les codes GPU et CPU	Mémoire constante GPU Durée de vie de l'application Ecrit par code CPU, lu par code GPU	Mémoire partagée d'un multiprocesseur Durée de vie du *block de threads* Accessible par le code GPU, sert à *cacher* la mémoire globale GPU

```
#include <cstdio>
#include "cuda.h"

#define N 1024
float A[N];
float c = 2.0;

__device__ float dA[N];

__global__ void multiplyArray(int n, float c)
{
  int elemParBlock = n / gridDim.x;
  int begin = blockIdx.x * elemParBlock;
  int end;
  if (blockIdx.x < gridDim.x - 1) {
    end = (blockIdx.x + 1) * elemParBlock;
  } else {
    end = n;
  }
  for (int i = begin; i < end; i++) { dA[i] *= c; }
}

int main(int argc, char **argv)
{
  // Initialisation
  for (int i = 0; i < N; i++) { A[i] = i; }
  // Copier le tableau vers le GPU
  cudaMemcpyToSymbol(dA, A, N * sizeof(float), 0,
      cudaMemcpyHostToDevice);
  multiplyArray<<<4, 1>>>(N, c);
  // Recopier le tableau multiplie vers le CPU
  cudaMemcpyFromSymbol(A, dA, N * sizeof(float), 0,
      cudaMemcpyDeviceToHost);
  printf("%lf\n", A[2]);
  return 0;
}
```

Source files must have the extension .cu (e.g., program.cu)

Compilation: nvcc program.cu-o program

Possible to specify the target architecture with-arch sm xx (e.g.-arch sm 75 for Turing)

Execution: ./program

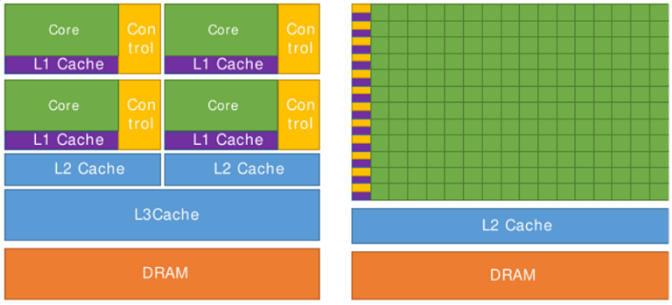
- __device__ defines an array on the GPU
- __global__ defines a function for the GPU.
  - Which enables using **blockIdx.x** and **gridDim.x** for example
- Without these keywords, it is for the CPU by default.
- We must copy data in the GPU memory before and after the computation with **cudaMemcpy...** (to come).
- Each block still executes the same code.
- Execution is differentiated by **blockIdx.x**
- With P blocks, each block iterates over N/P consecutive elements of **A[N]**.
- Careful for the last block if N is not divisible by P!

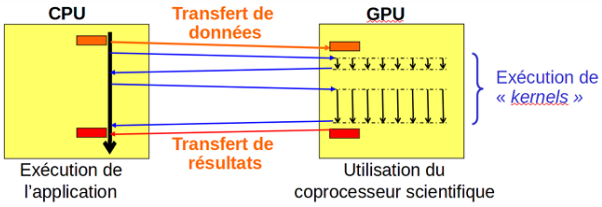
```
#include <cstdio>
#include "omp.h"

#define N 1024

int main(int argc, char **argv)
{
  float A[N];
  float c = 2.0;
  // Initialisation
  for (int i = 0; i < N; i++) { A[i] = i; }
#pragma omp parallel num_threads(4)
  {
    int thid = omp_get_thread_num();
    int numth = omp_get_num_threads();
    int elemParTh = N / numth;
    int begin = thid * elemParTh;
    int end;
    if (thid < numth - 1) { // Avant le dernier thr
      end = (thid + 1) * elemParTh;
    } else { // Le dernier thread
      end = N;
    }
    for (int i = begin; i < end; i++) {
      A[i] *= c;
    }
  }
  return 0;
}
```

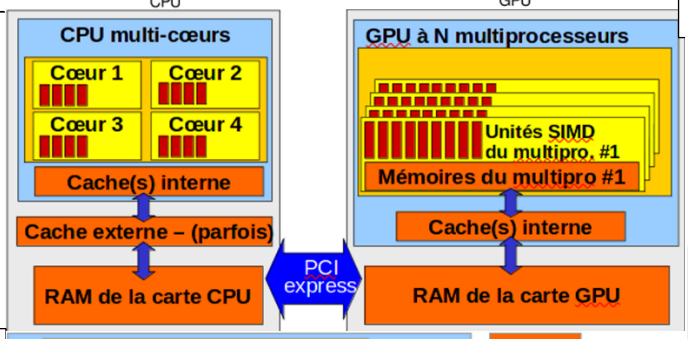
A comparison of CPU and GPU architecture : L1 cache potentially usable explicitly (shared memory) // L2 cache exists // No L3 cache // Few complex control circuits, notably (Out-of-order execution / Branch prediction // Instruction-level parallelism (ILP) // Complex instruction decoder) // 10x (or more) more computational power for the same chip area

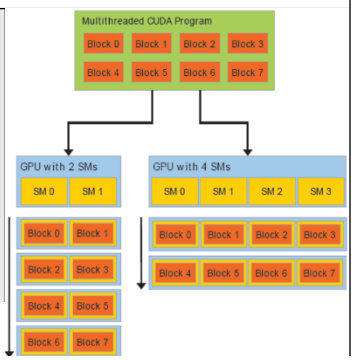




The program mainly runs on the CPU with calls to GPU functions : Before/after launching a kernel, data must be transferred // Data transfers should be minimized for efficiency // Each kernel call is non-blocking (i.e., the CPU continues execution), but it can be made blocking if desired

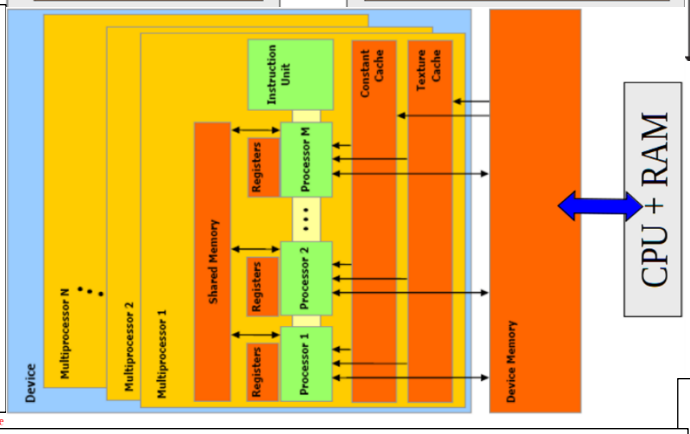
Overview of a CPU with a GPU : The CPU uses the GPU as a scientific coprocessor for certain calculations suited to the SIMD paradigm. // Both the CPU and GPU are multi-core and vector processors with a specific memory hierarchy. // Data transfer occurs over the PCI Express bus (32 GB/s bandwidth in each direction for PCIe4). // They do not have direct access to each other's RAM.

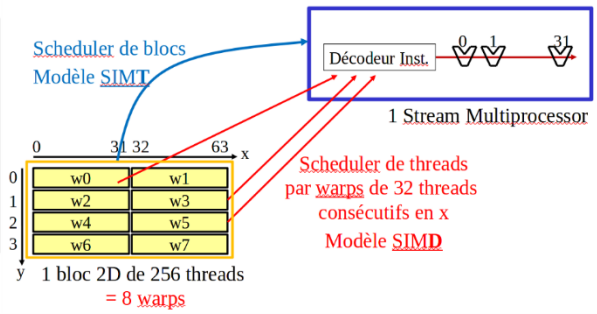




The CPU launches the execution of a kernel with a set of GPU threads : The block scheduler distributes the blocks across the different SMs with dynamic scheduling. // GPUs with different architectures can execute the same grid of thread blocks without any problem (with a distribution specific to their architecture, managed by the scheduler).

A GPU is a collection of N independent SIMD processors sharing a global memory : N streaming "multiprocessors" (SM) // Each SM is a SIMD processor having : k synchronized processors (k = 32), in other words, GPU cores // 1 shared instruction decoder // 3 types of memories shared among all k processors // 32k −128K registers distributed among the processors (63-255 specific to each thread) // To fully utilize each SM, you need to launch at least 32 threads (per block)





Certaines capacités de calculs n'évoluent pas de manière monotone

How many blocks/grid and threads/block should I use? : The thread scheduler wants to have many warps of threads in reserve to hide the memory access latency // The block scheduler wants to have many not-too-large blocks in order to (have blocks in reserve to use all SMs // overlap memory access times between blocks (an SM can host multiple blocks depending on resource availability (registers, shared memory, etc.) for better SM utilization // In general, 128-256 threads/block works well (min=1, max=1024). Optimal choice by experimentation or through an Nvidia tool

Create blocks with a certain number of warps The CPU launches the execution of a kernel with a set of GPU threads : An instruction decoder drives 32 threads hardware (32 GPU cores) // Each group of 32 consecutive threads in a block is called a warp // The scheduler executes each warp of an active block in an SM

Masking of memory access time of warps : The GPU switches from one warp to another very quickly (because they coexist physically in the SM) // The GPU masks memory access latency through multi-threading. // So do not hesitate to create a large number of small GPU threads per block and a large number of blocks (that is, little work per each "light" thread).

```
#define N 1024

// Static global array on the CPU
float ArrCPU[N];

// Static global array on the GPU
__device__ float ArrGPU[N];

// Dynamic array on the CPU
float *ArrCPU = (float *) malloc(N * sizeof(float));

// Dynamic array on the GPU
float *ArrGPU;
cudaError_t cuStat;
cuStat = cudaMalloc((void **) &ArrGPU, N * sizeof(float))
```

Allocation of an array on GPU : The prefix __device__ differentiates the declaration of a static GPU array from a CPU array. // The static GPU array must be declared outside of functions (as global variables) // Dynamic arrays on GPU are allocated using the cudaMalloc function.

Multiply each element of an array A[N] by a scalar c : global defines the function on the GPU.(This allows using blockIdx.x, blockDim.x, and threadIdx.x) // Each thread and block always executes the same code. // We use blockSize threads per block. // Each thread performs 1 operation, so N / blockSize blocks must be launched to cover the entire array/computation domain. // Execution is differentiated by blockIdx.x and threadIdx.x. // Be careful of array overflow (if N is not divisible by blockSize) ce code juste en bas

```
#define N 1024

// Static array on the CPU
float ArrCPU[N];

// Static array on the GPU
__device__ float ArrGPU[N];

cudaError_t cuStat;

// Copy a static CPU array onto a static GPU array
cuStat = cudaMemcpyToSymbol(ArrGPU, ArrCPU,
    sizeof(float) * N, 0, cudaMemcpyHostToDevice);

// Copy a static GPU array onto a static CPU array
cuStat = cudaMemcpyFromSymbol(ArrCPU, ArrGPU,
    sizeof(float) * N, 0, cudaMemcpyDeviceToHost);


// Copying a dynamic array between CPU and GPU
float *ArrCPU;
float *ArrGPU;
ArrCPU = (float *) malloc (N * sizeof(float));
cudaError_t cuStat;
cuStat = cudaMalloc((void **) &ArrGPU, N * sizeof(float))


// Copy a dynamic CPU array onto a dynamic GPU array
cudaStat = cudaMemcpy(ArrGPU, ArrCPU, sizeof(float)*N,
    cudaMemcpyHostToDevice);


// Copy a dynamic GPU array onto a dynamic CPU array
cudaStat = cudaMemcpy(ArrCPU, ArrGPU, sizeof(float)*N,
    cudaMemcpyDeviceToHost);
```

Copying a static array between a CPU and GPU

Multiply each element of an array A[N] by a scalar c : global defines the function on the GPU.(This allows using blockIdx.x) // Each block always executes the same code.// Each block performs 1 operation, so N blocks must be launched to cover the entire array/computation domain. // Execution is differentiated by blockIdx.x. (code en bas)

```
#include <cstdio>
#include "cuda.h"

#define N 1024
float A[N];
float c = 2.0;

__device__ float dA[N];

__global__ void multiplyArray(int n, float c)
{
  int i = blockIdx.x;
  dA[i] *= c;
}

int main(int argc, char **argv)
{
  // Initialisation
  for (int i = 0; i < N; i++) { A[i] = i; }
  // Copier le tableau vers le GPU
  cudaMemcpyToSymbol(dA, A, N * sizeof(float), 0,
      cudaMemcpyHostToDevice);
  multiplyArray<<<N, 1>>>(n, c);
  // Recopier le tableau multiplie vers le CPU
  cudaMemcpyFromSymbol(A, dA, N * sizeof(float), 0
      cudaMemcpyDeviceToHost);
  printf("%lf\n", A[2]);
  return 0;
}
```

```
#include <cstdio>
#include "cuda.h"

#define N 1024
float A[N];
float c = 2.0;

__device__ float dA[N];

__global__ void multiplyArray(int n, float c)
{
  int i = threadIdx.x + blockIdx.x * blockDim.x;
  if (i <  n)
  dA[i] *= c;
}

int main(int argc, char **argv)
{
  // Initialisation
  for (int i = 0; i < N; i++) { A[i] = i; }
  // Copier le tableau vers le GPU
  cudaMemcpyToSymbol(dA, A, N * sizeof(float), 0,
      cudaMemcpyHostToDevice);
  int blockSize = 128;
  int numBlocks = N / blockSize;
  if (N % blockSize) numBlocks++;
  multiplyArray<<<(numBlocks, blockSize>>>(n, c);
  // Recopier le tableau multiplie vers le CPU
  cudaMemcpyFromSymbol(A, dA, N * sizeof(float), 0,
      cudaMemcpyDeviceToHost);
  printf("%lf\n", A[2]);
  return 0;
}
```